

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Naveen Kumar S	Reg.No.:	192421030
Department:	B Tech IT	Date:	29th July 2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I Naveen Kumar S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Naveen Kumar S

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1.

Introduction

A processor represents signed integers using the **2's complement** method because it simplifies arithmetic operations. In an 8-bit system, signed integers range from **-128 to +127**. The most significant bit (MSB) acts as the **sign bit**, where **0** represents a positive number and **1** represents a negative number.

Step 1: Binary Conversion

Convert +45 to 8-bit Binary

Decimal value:

+45

Binary representation:

00101101

Convert -23 to 8-bit Binary

First, convert +23 into binary.

+23 = 00010111

Find the 1's complement:

11101000

Add 1 to obtain the 2's complement.

11101000

+00000001

11101001

Therefore,

-23 = 11101001

Operation 1: Addition of +45 and -23

Binary Representation

+45 = 00101101

-23 = 11101001

Binary Addition

```
  00101101
+ 11101001
-----
 1 00010110
```

The carry beyond the eighth bit is discarded.

Final 8-bit result:

00010110

Decimal Interpretation

$00010110_2 = 22_{10}$

Hence,

$45 + (-23) = 22$

Overflow Analysis for Addition

Overflow in signed 2's complement addition occurs only when:

- Two positive numbers produce a negative result, or
- Two negative numbers produce a positive result.

In this operation:

- +45 has sign bit **0** (positive).
- -23 has sign bit **1** (negative).

Since the operands have different signs, overflow cannot occur.

Another method of verification:

- Carry into the MSB = 1
- Carry out of the MSB = 1

Since both carries are equal,

Overflow Flag = 0

Therefore, **no overflow occurs.**

Operation 2: Subtraction of +45 and -23

Subtraction is performed by adding the 2's complement of the second operand.

$$\begin{aligned} &[\\ A - B &= A + (\text{2's complement of } B) \\ &] \end{aligned}$$

Since

$$\begin{aligned} &[\\ 45 - (-23) &= 45 + 23 \\ &] \end{aligned}$$

Convert +23 to binary.

00010111

Binary Addition

$$\begin{array}{r} 00101101 \\ + 00010111 \\ \hline 01000100 \end{array}$$

Final binary result:

01000100

Decimal Interpretation

01000100₂

$$= 64 + 4$$

$$= 68$$

Therefore,

$$45 - (-23) = 68$$

Overflow Analysis for Subtraction

The processor performs subtraction as an addition operation.

The operands being added are:

- +45 (positive)
- +23 (positive)

The result is:

- +68 (positive)

Since the result lies within the valid 8-bit signed range (**-128 to +127**), no overflow occurs.

Carry into the MSB = 0

Carry out of the MSB = 0

Therefore,

Overflow Flag = 0

Sign Bit Alignment

Before performing arithmetic, both operands are represented as 8-bit numbers.

$$+45 = 00101101$$

$$-23 = 11101001$$

The leftmost bit is the sign bit.

- **0** indicates a positive number.
- **1** indicates a negative number.

The processor aligns all eight bits, including the sign bit, and performs addition simultaneously using the binary adder.

How the Processor Detects Overflow

A processor detects signed overflow by comparing the carry entering and leaving the sign bit.

The overflow condition is given by:

$$[\text{Overflow}] = \text{Carry into MSB} \oplus \text{Carry out of MSB}$$

Overflow occurs when:

- Two positive operands produce a negative result.
- Two negative operands produce a positive result.

No overflow occurs when:

- The operands have opposite signs.
 - The carry into and carry out of the sign bit are identical.
-

Final Results

Operation	Binary Result	Decimal Result	Overflow
+45 + (-23)	00010110	22	No
+45 - (-23)	01000100	68	No

Conclusion

The processor successfully performs both addition and subtraction using **2's complement arithmetic**. The numbers **+45** and **-23** are first converted into 8-bit binary format. During addition, the processor adds the binary values directly, while subtraction is performed by adding the 2's complement of the second operand. In both operations, the final results (**22** and **68**) are within the valid 8-bit signed range (**-128 to +127**). Since the carry into and carry out of the sign bit are equal and the sign rules for overflow are not violated, **no overflow occurs**. This demonstrates the efficiency and reliability of 2's complement representation for signed arithmetic in modern processors.

2.

Introduction

Binary addition is one of the most frequently executed operations in a processor. The speed of arithmetic operations directly affects the overall performance of a computer system. A **Ripple Carry Adder (RCA)** is simple to design, but its delay increases with the number of bits because each carry must wait for the previous carry to be generated. To overcome this limitation, high-performance processors use a **Carry Look-Ahead Adder (CLA)**, which computes carry signals in advance using logic circuits. This significantly reduces propagation delay and increases processing speed.

Given Binary Inputs

Consider the following 4-bit binary numbers:

A = 1011

B = 0110

Carry input:

C₀ = 0

Step 1: Generate and Propagate Signals

For each bit position:

- **Generate Signal (G):**

$$\begin{aligned} &[\\ &G_i = A_i \times B_i \\ &] \end{aligned}$$

A carry is generated whenever both input bits are 1.

- **Propagate Signal (P):**

$$\begin{aligned} &[\\ &P_i = A_i \oplus B_i \\ &] \end{aligned}$$

A carry is propagated when exactly one input bit is 1.

The bit values are shown below.

Bit Position	A	B	Generate (G = A·B)	Propagate (P = A ⊕ B)
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Therefore,

$$G_3 \ G_2 \ G_1 \ G_0 = 0 \ 0 \ 1 \ 0$$

$$P_3 \ P_2 \ P_1 \ P_0 = 1 \ 1 \ 0 \ 1$$

Step 2: Carry Computation

Instead of waiting for carries one by one, the CLA computes every carry simultaneously.

The carry equations are:

$$\begin{bmatrix} C_1 = G_0 + P_0C_0 \end{bmatrix}$$

$$\begin{bmatrix} C_2 = G_1 + P_1G_0 + P_1P_0C_0 \end{bmatrix}$$

$$\begin{bmatrix} C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \end{bmatrix}$$

$$\begin{bmatrix} C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0 \end{bmatrix}$$

Substituting the values:

Carry C_1

$$C_1 = G_0 + P_0C_0$$

$$= 0 + (1 \times 0)$$

$$= 0$$

Carry C_2

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$= 1 + (0 \times 0) + (0 \times 1 \times 0)$$

$$= 1$$

Carry C_3

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$= 0 + (1 \times 1) + (1 \times 0 \times 0)$$

$$+ (1 \times 0 \times 1 \times 0)$$

$$= 1$$

Carry C_4

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1$$

$$+ P_3P_2P_1G_0$$

$$+ P_3P_2P_1P_0C_0$$

$$= 0 + (1 \times 0)$$

$$+ (1 \times 1 \times 1)$$

$$+ (1 \times 1 \times 0 \times 0)$$

$$+ (1 \times 1 \times 0 \times 1 \times 0)$$

$$= 1$$

Therefore,

$$C_1 = 0$$

$$C_2 = 1$$

$$C_3 = 1$$

$$C_4 = 1$$

Step 3: Sum Calculation

The sum bits are calculated as:

$$[S_i = P_i \oplus C_i]$$

Bit	Propagate (P)	Carry	Sum
S_0	1	0	1
S_1	0	0	0
S_2	1	1	0
S_3	1	1	0

Final result:

$$\text{Carry} = 1$$

$$\text{Sum} = 0001$$

Hence,

$$1011 + 0110 = 10001$$

Decimal verification:

$$1011_2 = 11$$

$$0110_2 = 6$$

$$11 + 6 = 17$$

$$10001_2 = 17$$

The binary addition is correct.

Comparison of CLA and Ripple Carry Adder

Ripple Carry Adder (RCA)

In a Ripple Carry Adder, each full adder must wait for the carry generated by the previous stage before producing its own carry.

For a 4-bit addition:

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$$

The carry "ripples" through every stage sequentially.

Time delay is approximately:

$$\begin{bmatrix} T_{\{RCA\}} = 4T_c \end{bmatrix}$$

where (T_c) is the delay of one carry stage.

Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder computes all carries simultaneously using generate and propagate logic.

$$C_1$$

C_2

C_3

C_4

(All computed in parallel)

The delay is approximately:

$$[T_{\{CLA\}} = T_G + T_P + T_{\{Logic\}}]$$

Since carries are generated simultaneously, the delay grows much more slowly than in a Ripple Carry Adder.

Time Delay Comparison

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry Generation	Sequential	Parallel
Delay	Increases with each bit	Nearly constant for small adders
Speed	Slower	Faster

Hardware
Complexity

Low

Higher

Suitable For

Small circuits

High-speed processors

Why CLA Improves Speed

A Carry Look-Ahead Adder improves processor performance because:

- It computes all carry bits simultaneously instead of sequentially.
- The propagation delay is greatly reduced.
- Faster arithmetic operations improve CPU execution speed.
- It is highly suitable for Arithmetic Logic Units (ALUs) in modern processors.
- It reduces bottlenecks during high-speed computations.
- Large processors implement hierarchical CLA structures to maintain fast arithmetic even for 32-bit and 64-bit operations.

Although the CLA requires additional logic gates and has a more complex design than the Ripple Carry Adder, the significant reduction in delay makes it the preferred choice for high-performance computing systems.

Conclusion

A **4-bit Carry Look-Ahead Adder (CLA)** enhances arithmetic performance by generating **generate (G)** and **propagate (P)** signals for each bit and computing all carry values in parallel. Unlike a Ripple Carry Adder, which waits for each carry to propagate sequentially, the CLA determines every carry simultaneously using combinational logic. This substantially reduces computation delay and enables faster binary addition. While the CLA requires more hardware, its superior speed and reduced propagation delay make it an essential component in modern processors and high-performance computing systems.

3.

Introduction

Booth's Algorithm is an efficient multiplication technique used in processors to perform multiplication of **signed binary numbers represented in 2's complement form**. Unlike conventional binary multiplication, Booth's Algorithm examines pairs of bits in the multiplier and performs addition or subtraction only when necessary. This reduces the total number of arithmetic operations, especially when the multiplier contains consecutive 1s, thereby improving processor performance.

Given Numbers

Multiplicand:

$$M = -6$$

Multiplier:

$$Q = +5$$

For this example, a **4-bit representation** is used.

Step 1: Convert Numbers to Binary

Multiplicand (-6)

First convert +6 to binary.

$$+6 = 0110$$

Find the 2's complement.

$$0110$$

↓ 1's complement

$$1001$$

+0001

1010

Therefore,

-6 = 1010

Multiplier (+5)

+5 = 0101

Step 2: Initialize Registers

The registers used are:

- **Accumulator (A)** = Stores intermediate results.
- **Multiplier (Q)** = Contains the multiplier.
- **Multiplicand (M)** = Contains the multiplicand.
- **Booth Bit (Q₋₁)** = Stores the previous least significant bit of the multiplier.
- **Count** = Number of bits (4).

Initial values:

Regist er	Value
--------------	-------

M	1010 (-6)
---	--------------

A	0000
---	------

Q 0101
 (+5)

Q_{-1} 0

Count 4

Booth's Decision Rules

At each iteration, the processor examines Q_0 (least significant bit of Q) and Q_{-1} .

Q_0 Q_{-1}	Operation
-------------------	-----------

00	No operation
----	-----------------

11	No operation
----	-----------------

01	$A = A + M$
----	-------------

10	$A = A - M$
----	-------------

After the selected operation, the registers **A**, **Q**, and Q_{-1} undergo one **arithmetic right shift**.

Step 3: Booth's Algorithm Execution

Iteration 1

Initial state:

$$A = 0000$$

$$Q = 0101$$

$$Q_{-1} = 0$$

$$Q_0Q_{-1} = \mathbf{10}$$

Perform:

$$A = A - M$$

$$0000 - 1010$$

$$= 0000 + 0110$$

$$= 0110$$

Arithmetic right shift:

Before Shift

$$A = 0110$$

$$Q = 0101$$

$$Q_{-1} = 0$$

After Shift

$A = 0011$

$Q = 0010$

$Q_{-1} = 1$

Iteration 2

Current state:

$A = 0011$

$Q = 0010$

$Q_{-1} = 1$

$Q_0Q_{-1} = \mathbf{01}$

Perform:

$A = A + M$

0011

+1010

1101

Arithmetic right shift:

Before Shift

$A = 1101$

$Q = 0010$

$Q_{-1} = 1$

After Shift

$A = 1110$

$Q = 1001$

$Q_{-1} = 0$

Iteration 3

Current state:

$A = 1110$

$Q = 1001$

$$Q_{-1} = 0$$

$$Q_0 Q_{-1} = \mathbf{10}$$

Perform:

$$A = A - M$$

$$1110 - 1010$$

$$= 1110 + 0110$$

$$= 0100$$

Arithmetic right shift:

Before Shift

$$A = 0100$$

$$Q = 1001$$

$$Q_{-1} = 0$$

After Shift

$$A = 0010$$

$$Q = 0100$$

$$Q_{-1} = 1$$

Iteration 4

Current state:

$$A = 0010$$

$$Q = 0100$$

$$Q_{-1} = 1$$

$$Q_0Q_{-1} = \mathbf{01}$$

Perform:

$$A = A + M$$

$$0010$$

$$+1010$$

$$1100$$

Arithmetic right shift:

Before Shift

$A = 1100$

$Q = 0100$

$Q_{-1} = 1$

After Shift

$A = 1110$

$Q = 0010$

$Q_{-1} = 0$

Count becomes zero.

Algorithm terminates.

Final Result

The final product is obtained by concatenating **A** and **Q**.

$AQ = 11100010$

This is an 8-bit 2's complement number.

To verify:

Take the 2's complement.

11100010

↓

00011101

+1

00011110

$00011110_2 = 30_{10}$

Since the original binary number begins with **1**, it represents a negative value.

Therefore,

$11100010_2 = -30_{10}$

Verification:

$$-6 \times 5 = -30$$

Hence, the multiplication is correct.

Role of Registers

Accumulator (A)

- Stores intermediate partial products.
- Performs addition and subtraction with the multiplicand.
- Forms the higher half of the final product.

Multiplier (Q)

- Holds the multiplier.
- Supplies the least significant bit for Booth's decision.
- Forms the lower half of the final product.

Booth Bit (Q_{-1})

- Stores the previous least significant bit.
 - Helps detect transitions between 0 and 1.
 - Determines whether addition, subtraction, or no operation is required.
-

Role of Arithmetic Right Shift

After every iteration, the combined register **A-Q- Q_{-1}** is shifted right.

The arithmetic shift:

- Preserves the sign bit.
- Maintains the correct signed value.
- Prepares the registers for the next iteration.

Unlike a logical shift, the arithmetic shift copies the sign bit into the new most significant position.

Why Booth's Algorithm is Efficient

Booth's Algorithm improves multiplication efficiency because:

- It directly supports signed numbers represented in 2's complement.
- Consecutive 1s in the multiplier are processed as a single addition and a single subtraction instead of multiple additions.
- It significantly reduces the number of arithmetic operations.
- It minimizes hardware complexity for signed multiplication.
- It is widely used in processor Arithmetic Logic Units (ALUs) and Digital Signal Processors (DSPs).

For example, a multiplier such as **11110000** would require many additions in conventional multiplication, whereas Booth's Algorithm replaces them with only one subtraction and one addition, reducing computation time.

Comparison with Traditional Binary Multiplication

Feature	Traditional Multiplication	Booth's Algorithm
Signed Numbers	Requires separate sign handling	Directly supports signed numbers
Addition Operations	One for every 1 in multiplier	Fewer additions/subtractions
Consecutive 1s	Many repeated additions	Treated efficiently
Speed	Slower for signed multiplication	Faster
Hardware Efficiency	Moderate	High

Conclusion

Booth's Algorithm provides an efficient method for multiplying signed binary numbers using **2's complement representation**. During multiplication of **-6** and **+5**, the processor uses the **Accumulator (A)**, **Multiplier (Q)**, **Multiplicand (M)**, and **Booth Bit (Q₋₁)** to determine whether addition, subtraction, or no operation is required. After each operation, an arithmetic right shift preserves the sign while preparing the registers for the next cycle. The final result obtained is **11100010₂**, which represents **-30**, confirming the correctness of the multiplication. Compared to traditional binary multiplication, Booth's Algorithm performs fewer arithmetic operations, handles signed numbers naturally, and offers higher efficiency, making it well suited for modern processors.

4.

Introduction

Division is one of the fundamental arithmetic operations performed by a processor. In digital systems, binary division is implemented using hardware algorithms instead of repeated subtraction. Two commonly used methods are:

1. **Restoring Division Algorithm**
2. **Non-Restoring Division Algorithm**

Both algorithms use registers to store the dividend, divisor, quotient, and remainder. The primary difference is that the restoring algorithm restores the previous remainder whenever subtraction results in a negative value, whereas the non-restoring algorithm avoids this restoration, thereby reducing the number of arithmetic operations.

Given Binary Numbers

Dividend:

$$13_{10} = 1101_2$$

Divisor:

$$3_{10} = 0011_2$$

Expected Result:

$$13 \div 3 = 4$$

Remainder = 1

Registers Used

Register	Purpose
A	Accumulator (stores remainder)
Q	Dividend / Quotient register
M	Divisor
Count	Number of bits (4)

Initial values:

A = 0000

Q = 1101

M = 0011

Count = 4

Method 1: Restoring Division Algorithm

Working Principle

Each iteration performs the following operations:

1. Shift the combined register (A,Q) left.

2. Subtract the divisor from A.
 3. If the result is positive, set the quotient bit to 1.
 4. If the result is negative, restore A by adding the divisor back and set the quotient bit to 0.
 5. Repeat until all bits are processed.
-

Iteration 1

After left shift:

A = 0001

Q = 1010

Subtract divisor:

0001

-0011

1110 (Negative)

Restore remainder:

1110

+0011

0001

Quotient bit:

$Q_0 = 0$

Iteration 2

After left shift:

$A = 0011$

$Q = 0100$

Subtract divisor:

0011

-0011

0000

Result is non-negative.

Quotient bit:

$Q_0 = 1$

Iteration 3

After left shift:

$A = 0000$

Q = 1001

Subtract divisor:

0000

-0011

1101 (Negative)

Restore:

1101

+0011

0000

Quotient bit:

$Q_0 = 0$

Iteration 4

After left shift:

A = 0001

Q = 0010

Subtract divisor:

0001

-0011

1110 (Negative)

Restore:

1110

+0011

0001

Quotient bit:

$Q_0 = 0$

Final Result (Restoring Division)

Quotient = $0100_2 = 4$

Remainder = $0001_2 = 1$

Verification:

$$3 \times 4 + 1 = 13$$

Therefore, the result is correct.

Method 2: Non-Restoring Division Algorithm

Working Principle

The non-restoring division algorithm eliminates the restoration step.

The operations are:

- If the remainder is positive or zero, subtract the divisor.
- If the remainder is negative, add the divisor.
- Shift left after each iteration.
- The quotient bit depends on the sign of the current remainder.
- After all iterations, if the final remainder is negative, perform one final restoration.

This approach reduces the number of arithmetic operations.

Iteration 1

Initial values:

A = 0000

Q = 1101

Shift left:

A = 0001

Q = 1010

Subtract divisor:

0001

-0011

1110 (Negative)

Quotient bit:

0

Iteration 2

Since remainder is negative, add divisor.

1110

+0011

0001

Shift left:

A = 0011

Q = 0100

Subtract divisor:

0011

-0011

0000

Quotient bit:

1

Iteration 3

Shift left:

A = 0000

Q = 1001

Subtract divisor:

0000

-0011

1101 (Negative)

Quotient bit:

0

Iteration 4

Since remainder is negative, add divisor.

1101

+0011

0000

Shift left:

A = 0001

Q = 0010

Subtract divisor:

0001

-0011

1110 (Negative)

Quotient bit:

0

The final remainder is negative.

Perform one final restoration.

1110

+0011

0001

Final Result (Non-Restoring Division)

Quotient = $0100_2 = 4$

Remainder = $0001_2 = 1$

Verification:

$$3 \times 4 + 1 = 13$$

The result is correct.

Comparison of Restoring and Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Restoration Step	Required whenever remainder becomes negative	Not required during every iteration
Arithmetic Operations	More	Fewer
Hardware Complexity	Moderate	Slightly higher control logic
Execution Speed	Slower	Faster
Efficiency	Lower	Higher
Processor Usage	Older processors	Modern processors

Why Non-Restoring Division is Preferred

Non-restoring division is widely used in modern processors because:

- It eliminates repeated restoration operations.
- Fewer additions and subtractions reduce execution time.
- It requires fewer arithmetic operations overall.
- It improves processor throughput.
- It is well suited for high-speed Arithmetic Logic Units (ALUs).
- It provides better performance for large binary numbers and high-performance computing applications.

Although its control logic is slightly more complex than restoring division, the reduction in arithmetic operations makes it significantly more efficient.

Time and Complexity Analysis

Algorithm	Arithmetic Operations	Relative Speed
Restoring Division	More due to restoration	Slower
Non-Restoring Division	Fewer because restoration is avoided	Faster

For an **n-bit dividend**, both algorithms require approximately **n iterations**, but the restoring algorithm may perform additional restoration additions whenever the remainder becomes negative. The non-restoring algorithm avoids these repeated restorations, making it more efficient in practical processor implementations.

Conclusion

Both restoring and non-restoring division algorithms correctly compute the division of **13 by 3**, producing a **quotient of 4** and a **remainder of 1**. The restoring division algorithm restores the previous remainder whenever subtraction results in a negative value, increasing the number of arithmetic operations. In contrast, the non-restoring division algorithm avoids immediate restoration by using the sign of the current remainder to determine the next operation and performs only one final correction if necessary. As a result, non-restoring division reduces computation time, improves execution efficiency, and is therefore the preferred division technique in modern processors and digital systems.

5.

Introduction

Scientific and engineering applications often require the representation of very large or very small real numbers with high precision. Modern processors use the **IEEE 754 floating-point standard** to represent and manipulate real numbers. This standard provides a uniform format for storing floating-point values and defines how arithmetic operations such as addition, subtraction, multiplication, and division are performed.

The IEEE 754 standard represents a floating-point number using three fields:

- **Sign Bit (S)** – Indicates whether the number is positive or negative.
 - **Exponent (E)** – Stores the exponent using a bias value.
 - **Mantissa (Fraction or Significand)** – Stores the significant digits of the number.
-

Given Decimal Number

[
-13.25₁₀
]

Step 1: Convert the Decimal Number to Binary

Convert the integer part:

$$13_{10} = 1101_2$$

Convert the fractional part:

$$0.25 \times 2 = 0.5 \rightarrow 0$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

Therefore,

$$13.25_{10} = 1101.01_2$$

Hence,

$$-13.25_{10} = -1101.01_2$$

Step 2: Normalize the Binary Number

Move the binary point so that only one digit remains before the decimal point.

$$1101.01_2$$

=

$$1.10101_2 \times 2^3$$

Therefore,

- **Normalized Mantissa = 1.10101**
 - **Exponent = 3**
-

IEEE 754 Single Precision (32-bit)

The single precision format consists of:

Field	Number of Bits
-------	----------------

Sign	1
------	---

Exponent 8

Mantissa 23

Sign Bit

Since the number is negative,

Sign = 1

Exponent

Bias for single precision:

127

Stored exponent:

$127 + 3 = 130$

Binary representation:

$130 = 10000010_2$

Mantissa

The leading 1 is omitted because it is implicit.

Normalized value:

1.10101

Stored mantissa:

101010000000000000000000

(23 bits)

Final 32-bit Representation

Sig n	Expone nt	Mantissa
1	100000 10	101010000000000000 000000

Complete binary representation:
1 10000010 101010000000000000000000

IEEE 754 Double Precision (64-bit)

The double precision format consists of:

Field	Number of Bits
Sign	1
Expon ent	11
Mantis sa	52

Sign Bit

1

Exponent

Bias for double precision:

1023

Stored exponent:

$$1023 + 3 = 1026$$

Binary representation:

$$10000000010_2$$

Mantissa

1010100

(52 bits)

Final 64-bit Representation

Sig	Exponent	Mantissa
-----	----------	----------

n

[illegible]

Complete representation:

1 10000000010

1010100

Comparison of Single and Double Precision

Feature	Single Precision	Double Precision
Total Bits	32	64
Sign Bits	1	1
Exponent Bits	8	11
Mantissa Bits	23	52
Bias	127	1023
Approximate Decimal Precision	7 digits	15–16 digits

Floating-Point Addition

Consider the addition of two floating-point numbers:

13.25 + 2.5

Step 1: Convert to Normalized Form

13.25

=

$$1.10101 \times 2^3$$

$$2.5$$

=

$$1.01 \times 2^1$$

Step 2: Align the Exponents

The larger exponent is **3**.

Shift the mantissa of the second number to match the exponent.

$$1.01 \times 2^1$$

↓

$$0.0101 \times 2^3$$

Now both numbers have the same exponent.

Step 3: Add the Mantissas

$$1.10101$$

$$+0.01010$$

1.11111

Exponent remains:

2^3

Step 4: Normalize the Result

The result is already normalized.

1.11111×2^3

No further shifting is required.

Step 5: Rounding

If the mantissa exceeds the available number of bits:

- Extra bits are removed.
- IEEE 754 typically uses **Round to Nearest, Even**.
- This minimizes cumulative rounding errors.

For this example, no rounding is required because the result fits within the available precision.

Normalization

Normalization ensures that the binary number is stored in the form:

$1.xxxxx \times 2^n$

Benefits include:

- Maximizes precision.
- Provides a unique representation.
- Makes arithmetic operations more efficient.

Rounding

Sometimes a floating-point result contains more bits than can be stored.

Example:

1.101010111100...

If only a limited number of mantissa bits are available, the remaining bits are rounded according to the IEEE 754 rounding rules.

Common rounding modes include:

- Round to Nearest (Even)
- Round Toward Zero
- Round Toward Positive Infinity
- Round Toward Negative Infinity

The default mode is **Round to Nearest, Even**.

Precision Loss

Floating-point numbers cannot represent every real number exactly.

For example,

0.1_{10}

cannot be represented exactly in binary.

Instead, the processor stores an approximation.

Repeated arithmetic operations may introduce small rounding errors.

This phenomenon is called **precision loss**.

Examples of precision loss include:

- Repeated addition of small values.
- Subtraction of nearly equal numbers (cancellation error).
- Limited mantissa size causing truncation.

Double precision significantly reduces these errors because it stores more mantissa bits than single precision.

Advantages of Double Precision

- Greater numerical accuracy.
- Higher precision (approximately 15–16 decimal digits).
- Reduced rounding errors.
- Better suited for scientific simulations, engineering calculations, and machine learning applications.

The trade-off is that double precision requires more memory and slightly more computation than single precision.

Conclusion

The IEEE 754 standard provides a reliable and consistent method for representing real numbers in computer systems. The decimal number **–13.25** is represented by dividing it into **sign**, **exponent**, and **mantissa** fields in both **single precision (32-bit)** and **double precision (64-bit)** formats. During floating-point addition, the processor first aligns the exponents, adds the mantissas, normalizes the result, and applies rounding when necessary. Although floating-point arithmetic offers a wide dynamic range, finite mantissa size can lead to **rounding errors** and **precision loss**. Double precision minimizes these errors by providing a larger exponent range and greater mantissa precision, making it the preferred choice for scientific and high-accuracy computational applications.

